

# BÀI TẬP 3

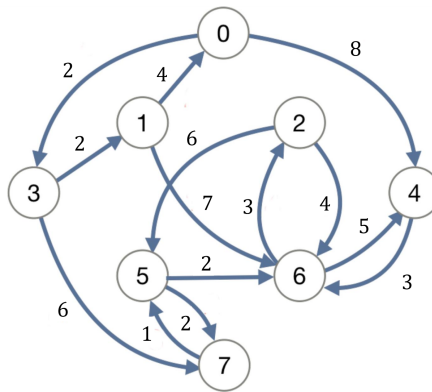
Nhóm 5

Trần Anh Minh-21000242

Lê Thị Hương-21000684

Lê Quốc Khánh-21002466

## Bài tập 1:



## Thuật toán Dijkstra

Các ô trong bảng biểu thị khoảng cách ngắn nhất tại mỗi bước từ đỉnh gốc (đỉnh ①) đến các đỉnh tương ứng ở từng cột. Đồng thời, mỗi ô cũng ghi lại đỉnh liền trước mà từ đó ta đi đến đỉnh hiện tại trên đường đi ngắn nhất.

| Đỉnh<br>Bước | ①      | ②       | ③       | ④       | ⑤       | ⑥               | ⑦       |
|--------------|--------|---------|---------|---------|---------|-----------------|---------|
| Khởi tạo     | (①, 0) | (_, +∞) | (_, +∞) | (_, +∞) | (_, +∞) | (_, +∞)         | (_, +∞) |
| 1            |        | (_, +∞) | (_, +∞) | (①, 2)  | (①, 8)  | (_, +∞)         | (_, +∞) |
| 2            |        | (③, 4)  | (_, +∞) |         | (①, 8)  | (_, +∞)         | (③, 8)  |
| 3            |        |         | (_, +∞) |         | (①, 8)  | (①, 11)         | (③, 8)  |
| 4            |        |         | (_, +∞) |         |         | ({①, ④}, 11)    | (③, 8)  |
| 5            |        |         | (_, +∞) |         | (⑦, 9)  | ({①, ④}, 11)    |         |
| 6            |        |         | (_, +∞) |         |         | ({①, ④, ⑤}, 11) |         |
| 7            |        |         | (⑥, 14) |         |         |                 |         |
| 8            |        |         |         |         |         |                 |         |

Vậy đường đi ngắn nhất từ ① đến ② có trọng số là 14 và là các đường sau:

- ① → ③ → ⑦ → ⑤ → ⑥ → ②
- ① → ④ → ⑥ → ②
- ① → ③ → ① → ⑥ → ②

## Thuật toán Bellman-Ford

Khởi tạo:

| Đỉnh       | ①         | ②         | ③    | ④         | ⑤         | ⑥         | ⑦         |
|------------|-----------|-----------|------|-----------|-----------|-----------|-----------|
| distances  | $+\infty$ | $+\infty$ | 0    | $+\infty$ | $+\infty$ | $+\infty$ | $+\infty$ |
| successors | null      | null      | null | null      | null      | null      | null      |

Lần lặp 1:

- Xét các cạnh lân cận của đỉnh ②

| Đỉnh       | ①         | ②         | ③    | ④         | ⑤         | ⑥ | ⑦         |
|------------|-----------|-----------|------|-----------|-----------|---|-----------|
| distances  | $+\infty$ | $+\infty$ | 0    | $+\infty$ | $+\infty$ | 3 | $+\infty$ |
| successors | null      | null      | null | null      | null      | ② | null      |

Lần lặp 2:

- Xét các cạnh lân cận của đỉnh ⑥

| Đỉnh       | ①         | ②  | ③    | ④         | ⑤ | ⑥ | ⑦ |
|------------|-----------|----|------|-----------|---|---|---|
| distances  | $+\infty$ | 10 | 0    | $+\infty$ | 6 | 5 | 3 |
| successors | null      | ⑥  | null | null      | ⑥ | ⑥ | ② |

Lần lặp 3:

- Xét các cạnh lân cận của đỉnh ①

| Đỉnh       | ①         | ②  | ③    | ④  | ⑤ | ⑥ | ⑦ |
|------------|-----------|----|------|----|---|---|---|
| distances  | $+\infty$ | 10 | 0    | 12 | 6 | 5 | 3 |
| successors | null      | ⑥  | null | ①  | ⑥ | ⑥ | ② |

- Xét các cạnh lân cận của đỉnh ④

| Đỉnh       | ①  | ②  | ③    | ④  | ⑤ | ⑥ | ⑦ |
|------------|----|----|------|----|---|---|---|
| distances  | 16 | 10 | 0    | 12 | 6 | 5 | 3 |
| successors | ④  | ⑥  | null | ①  | ⑥ | ⑥ | ② |

- Xét các cạnh lân cận của đỉnh ⑤

| Đỉnh       | ①  | ②  | ③    | ④  | ⑤ | ⑥ | ⑦ |
|------------|----|----|------|----|---|---|---|
| distances  | 16 | 10 | 0    | 12 | 6 | 5 | 3 |
| successors | ④  | ⑥  | null | ①  | ⑥ | ⑥ | ② |

### Lần lặp 4:

- Xét các cạnh lân cận của đỉnh ③

| Đỉnh       | ①  | ②  | ③    | ④  | ⑤ | ⑥ | ⑦ |
|------------|----|----|------|----|---|---|---|
| distances  | 16 | 10 | 0    | 12 | 6 | 5 | 3 |
| successors | ④  | ⑥  | null | ①  | ⑥ | ⑥ | ② |

- Xét các cạnh lân cận của đỉnh ①

| Đỉnh       | ①  | ②  | ③    | ④  | ⑤ | ⑥ | ⑦ |
|------------|----|----|------|----|---|---|---|
| distances  | 14 | 10 | 0    | 12 | 6 | 5 | 3 |
| successors | ③  | ⑥  | null | ①  | ⑥ | ⑥ | ② |

- Xét các cạnh lân cận của đỉnh ⑦

| Đỉnh       | ①  | ②  | ③    | ④  | ⑤ | ⑥ | ⑦ |
|------------|----|----|------|----|---|---|---|
| distances  | 14 | 10 | 0    | 12 | 6 | 5 | 3 |
| successors | ③  | ⑥  | null | ①  | ⑥ | ⑥ | ② |

### Lần lặp 5:

- Xét các cạnh lân cận của đỉnh ①

| Đỉnh       | ①  | ②  | ③    | ④  | ⑤ | ⑥ | ⑦ |
|------------|----|----|------|----|---|---|---|
| distances  | 14 | 10 | 0    | 12 | 6 | 5 | 3 |
| successors | ③  | ⑥  | null | ①  | ⑥ | ⑥ | ② |

**Nhận xét:** Lần lặp 5 không thay đổi gì so với lần lặp 3 nên ta dừng thuật toán.

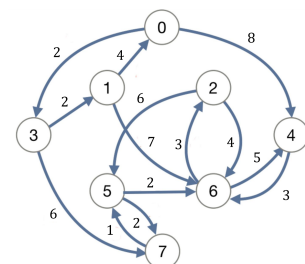
|            | Đường đi          | Tổng trọng số |
|------------|-------------------|---------------|
| Từ ① đến ② | ① → ③ → ① → ⑥ → ② | 14            |
| Từ ① đến ② | ① → ⑥ → ②         | 10            |
| Từ ③ đến ② | ③ → ① → ⑥ → ②     | 12            |
| Từ ④ đến ② | ④ → ⑥ → ②         | 6             |
| Từ ⑤ đến ② | ⑤ → ⑥ → ②         | 5             |
| Từ ⑥ đến ② | ⑥ → ②             | 3             |
| Từ ⑦ đến ② | ⑦ → ⑤ → ⑥ → ②     | 6             |

### Bài tập 2:

Đường đi ngắn nhất từ ① đến ② có trọng số là 14 và là các đường sau:

- ① → ③ → ⑦ → ⑤ → ⑥ → ②
- ① → ④ → ⑥ → ②
- ① → ③ → ① → ⑥ → ②

Vậy đường đi ngắn nhất trong một đồ thị không phải duy nhất.



## Bài tập 3:

### Ý tưởng thuật toán:

#### 1. Chạy Dijkstra từ đỉnh $s$ .

Khi gặp cạnh  $(u, v)$  ta sẽ coi trọng số của nó là 0, ta sử dụng thuật toán Dijkstra để tính khoảng cách ngắn nhất từ  $s$  đến tất cả các đỉnh.

Ta sẽ thu được mảng  $D_1[x]$  là khoảng cách ngắn nhất từ  $s$  đến các đỉnh  $x$  trong đồ thị. Đồng thời, ta cũng thu được mảng  $S_1[x]$ , trong đó  $S_1[x]$  là đỉnh trước đỉnh  $x$  trên đường đi ngắn nhất từ  $s$  đến  $x$  ( $x \in V$ ).

Nếu trong đường đi ngắn nhất của  $s$  đến  $t$  đã bao gồm cạnh  $(u, v)$  thì ta gán  $D_1[t] = D_1[u] + w(u, v)$  và dừng thuật toán ( $w(u, v)$  là trọng số của cạnh  $(u, v)$ ).

Nếu trong đường đi ngắn nhất của  $s$  đến  $t$  không bao gồm cạnh  $(u, v)$  thì ta tiếp tục các bước sau.

#### 2. Chạy Dijkstra từ đỉnh $v$ .

Ta áp dụng thuật toán Dijkstra một lần nữa với đỉnh nguồn  $v$ , ta thu được mảng  $D_2[x]$  và  $S_2[x]$  lần lượt chứa khoảng cách ngắn nhất từ  $v$  đến đỉnh  $x$  và đỉnh trước đỉnh  $x$  trên đường đi ngắn nhất từ  $v$  đến  $x$  ( $x \in V$ ).

#### 3. Tính toán đường đi ngắn nhất từ $s$ đến $t$ .

Ta có hai khả năng cho đường đi ngắn nhất từ  $s$  đến  $t$ :

- **Trường hợp 1:** Đường đi ngắn nhất từ  $s$  đến  $t$  không đi qua cạnh  $(u, v)$ . Khi đó, đường đi ngắn nhất là  $D_1[t]$ , tức là khoảng cách từ  $s$  đến  $t$  trong đồ thị đã loại bỏ cạnh  $(u, v)$ .
- **Trường hợp 2:** Đường đi ngắn nhất từ  $s$  đến  $t$  đi qua cạnh  $(u, v)$ . Khi đó, đường đi từ  $s$  đến  $t$  sẽ là đường từ  $s$  đến  $u$  rồi qua  $(u, v)$  và cuối cùng từ  $v$  đến  $t$ . Khoảng cách này được tính là:

$$D_1[u] + w(u, v) + D_2[t]$$

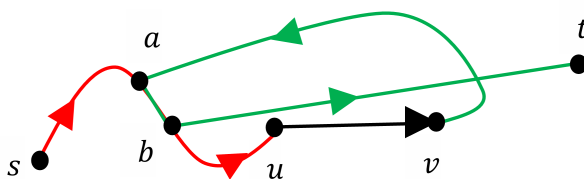
Đường đi ngắn nhất thực sự từ  $s$  đến  $t$  sẽ là giá trị nhỏ nhất giữa hai giá trị tính được từ hai trường hợp trên:

$$\text{shortest\_path}(s, t) = \min(D_1[t], D_1[u] + w(u, v) + D_2[t])$$

Ta không cần quan tâm đến việc đường đi ngắn nhất từ  $v$  đến  $t$  và đường đi từ  $s$  đến  $u$  bị trùng 1 số đỉnh và cạnh. Giả sử chúng có đường đi con từ  $a$  đến  $b$  như sau:

Đường đi ngắn nhất từ  $s$  đến  $u$ :  $s \rightarrow \dots \rightarrow a \rightarrow \dots \rightarrow b \rightarrow u$ .

Đường đi ngắn nhất từ  $v$  đến  $t$ :  $v \rightarrow \dots \rightarrow a \rightarrow \dots \rightarrow b \rightarrow t$ .



Ta gọi độ dài của đường đi  $s \rightarrow \dots \rightarrow a \rightarrow \dots \rightarrow b \rightarrow \dots \rightarrow t$  không chứa cạnh  $(u, v)$  là  $D_0$ . Do đồ thị không có chu trình âm mà đường đi  $s \rightarrow \dots \rightarrow a \rightarrow \dots \rightarrow b \rightarrow \dots \rightarrow t$  ít hơn đường đi  $s \rightarrow \dots \rightarrow u \rightarrow v \rightarrow \dots \rightarrow t$  một chu trình  $a \rightarrow \dots \rightarrow b \rightarrow \dots \rightarrow u \rightarrow v \rightarrow \dots \rightarrow a$  nên ta có:

$$D_1[t] \leq D_0 < D_1[u] + w(u, v) + D_2[t]$$

Vì vậy trong trường hợp này hàm `shortest_path` trên vẫn sẽ trả ra kết quả đúng là đường đi ngắn nhất không bao gồm cạnh  $(u, v)$ .

### Độ phức tạp thuật toán:

Áp dụng 2 lần thuật toán Dijkstra như trên sẽ có độ phức tạp là  $O((|E| + |V| \log |V|))$ , đây là độ phức tạp tốt nhất có thể đạt được cho bài toán này.

## Bài tập 4:

### 1. Seam Carving

Seam Carving là một kỹ thuật xử lý ảnh nhằm thay đổi kích thước hình ảnh mà vẫn giữ được các chi tiết quan trọng. Ta áp dụng bài toán đường đi ngắn nhất để tìm ra đường seam có tổng năng lượng thấp nhất trong ảnh. Sau khi tìm được đường seam, ta có thể loại bỏ hoặc thêm vào để điều chỉnh kích thước hình ảnh một cách hợp lý.



Hình ảnh trước khi Seam Carving



Hình ảnh sau khi Seam Carving

Năng lượng của một điểm ảnh được định nghĩa dựa trên sự biến đổi màu sắc tại điểm đó và các điểm lân cận. Một phương pháp để tính năng lượng là sử dụng đạo hàm Sobel. Công thức tính năng lượng tại điểm  $(i, j)$  như sau:

$$E(i, j) = \sqrt{(I_x(i, j))^2 + (I_y(i, j))^2}$$

trong đó,  $I_x$  và  $I_y$  là các độ biến thiên của cường độ điểm ảnh theo phương ngang và dọc. Nếu như ảnh có 3 kênh màu thì ta lấy tổng giá trị năng lượng của 3 kênh này lại với nhau.

Nếu loại bỏ các điểm ảnh có năng lượng thấp ngẫu nhiên, hình ảnh có thể bị méo mó. Tuy nhiên, việc xóa đường seam có tổng năng lượng thấp nhất theo chiều dọc hoặc ngang sẽ giúp giảm kích thước ảnh mà vẫn giữ được các chi tiết chính.

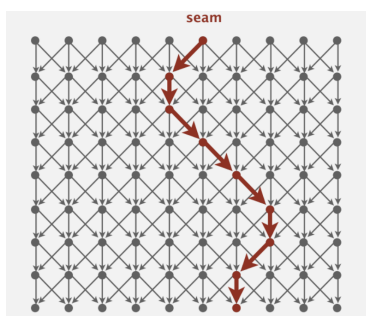
#### 1. Đường seam dọc:

- Là một đường đi từ biên trên xuống biên dưới của ảnh, với độ dài bằng chiều cao của ảnh.
- Tại mỗi vị trí  $(i, j)$ , điểm tiếp theo có thể là một trong ba điểm lân cận:  $(i + 1, j - 1)$ ,  $(i + 1, j)$ , hoặc  $(i + 1, j + 1)$ .

#### 2. Đường seam ngang:

- Được định nghĩa tương tự như đường seam dọc, nhưng chạy từ biên trái sang biên phải của ảnh.
- Tại mỗi vị trí  $(i, j)$ , điểm tiếp theo có thể là một trong ba điểm lân cận:  $(i - 1, j + 1)$ ,  $(i, j + 1)$ , hoặc  $(i + 1, j + 1)$ .

Trong mô hình đồ thị, mỗi điểm ảnh được xem như một đỉnh, và các đỉnh được nối với nhau bằng các cạnh tương ứng với các điểm ảnh liên kề mà đường seam có thể đi qua. Trọng số của mỗi cạnh chính là năng lượng của điểm ảnh mà nó dẫn đến. Quá trình tìm đường seam có tổng năng lượng thấp nhất chính là ứng dụng bài toán đường đi ngắn nhất trên đồ thị này.



Đường seam



Ví dụ các đường seam trong ảnh

## 2. Định hướng robot tự động

Trong việc định hướng robot tự động, thuật toán đường đi ngắn nhất đóng vai trò quan trọng trong việc xác định lộ trình tối ưu giữa các điểm di chuyển, đặc biệt trong các môi trường như nhà kho, nhà máy. Trong bài toán này, các đỉnh có thể đại diện cho các vị trí hoặc điểm đến mà robot cần di chuyển đến, chẳng hạn như các kệ hàng hoặc trạm sạc. Các cạnh thể hiện những đường kết nối giữa các đỉnh, tương ứng với các tuyến đường khả thi mà robot có thể đi qua. Mỗi cạnh có thể được gán một trọng số, thường là khoảng cách hoặc thời gian di chuyển. Robot tự động sẽ sử dụng thuật toán để tìm đường ngắn nhất giữa hai đỉnh (vị trí), đồng thời phải tránh các chướng ngại vật trên cạnh (đường đi). Nhờ vậy, robot có thể hoàn thành nhiệm vụ như lấy hàng và vận chuyển một cách hiệu quả, tối ưu hóa thời gian và giảm thiểu rủi ro va chạm trong quá trình hoạt động.

